

CSA-0606- DESIGN AND ANALYSIS OF ALGORITHMS

CO 2 – AT 3

Q71. Debugging Selection Sort Code

1. Objective

- Debug the Selection Sort program by correcting the inner loop.
- Ensure only the unsorted part of the array is searched.
- Avoid unnecessary swaps when the minimum element is already in the correct position.
- Improve efficiency while maintaining correct sorting.

2. Problem Identification

Given Incorrect Code

```
def selection_sort(arr):  
    n = len(arr)  
    for i in range(n):  
        min_idx = i  
        for j in range(0, n): # ERROR  
            if arr[j] < arr[min_idx]:  
                min_idx = j  
        arr[i], arr[min_idx] = arr[min_idx], arr[i]  
    return arr
```

Issues

- The inner loop starts from 0 instead of $i + 1$.
- Already sorted elements are searched again.
- Previously sorted elements may be disturbed.
- Unnecessary swaps are performed even when $\text{min_idx} == i$.

Impact

- Incorrect sorting.
- Extra comparisons.
- Extra swap operations.

3. Root Cause Analysis

The main error is:

for j in range(0, n):

It should only search the **unsorted portion** of the array.

Correct loop:

for j in range(i + 1, n):

Reason:

- Elements before index i are already sorted.
- Searching them again can select an element from the sorted portion and break the algorithm.

Another issue:

```
arr[i], arr[min_idx] = arr[min_idx], arr[i]
```

This swap happens every iteration.

It should only occur when

```
min_idx != i
```

4. Debugging Methodology

Step 1

Read the algorithm carefully.

Step 2

Compare it with the standard Selection Sort algorithm.

Step 3

Trace the program using a sample input.

Example:

[64, 25, 12, 22, 11]

Observe that the inner loop repeatedly checks already sorted elements.

Step 4

Correct the loop:

`range(i + 1, n)`

Step 5

Add a condition to avoid unnecessary swaps.

Step 6

Run multiple test cases.

5. Solution Implemented

Corrected and Optimized Code

```
def selection_sort(arr):  
    n = len(arr)  
  
    # Traverse through the array  
    for i in range(n - 1):  
  
        # Assume current index has minimum value  
        min_idx = i
```

```
# Search only the unsorted portion
for j in range(i + 1, n):

    if arr[j] < arr[min_idx]:
        min_idx = j

# Swap only if required
if min_idx != i:
    arr[i], arr[min_idx] = arr[min_idx], arr[i]

return arr
```

```
# Driver Code
numbers = [64, 25, 12, 22, 11]

print("Original Array :", numbers)

selection_sort(numbers)

print("Sorted Array :", numbers)
```

Output

```
Original Array : [64, 25, 12, 22, 11]
Sorted Array  : [11, 12, 22, 25, 64]
```

6. Optimization Techniques

1. Scan only the unsorted portion

Before:

```
range(0, n)
```

After:

```
range(i + 1, n)
```

Reduces unnecessary comparisons.

2. Avoid unnecessary swaps

Before:

```
arr[i], arr[min_idx] = arr[min_idx], arr[i]
```

After:

```
if min_idx != i:
```

```
    arr[i], arr[min_idx] = arr[min_idx], arr[i]
```

Reduces unnecessary data movement.

3. Reduce outer loop

Instead of

```
for i in range(n):
```

Use

```
for i in range(n - 1):
```

The last element is automatically sorted.

7. Performance Evaluation

Metric	Before Debugging	After Debugging
Correct Sorting	 No	 Yes
Comparisons	More (checks entire array every pass)	Fewer (checks only unsorted part)
Unnecessary Swaps	Yes	No
Execution Efficiency	Lower	Higher
Space Complexity	$O(1)$	$O(1)$

Time Complexity

Case	Complexity
Best Case	$O(n^2)$
Average Case	$O(n^2)$
Worst Case	$O(n^2)$

Space Complexity

$O(1)$

Selection Sort always performs nested-loop comparisons, so its time complexity remains **$O(n^2)$** . The optimization reduces unnecessary work but does not change the asymptotic complexity.

8. Conclusion

The Selection Sort implementation was successfully debugged by correcting the inner loop to scan only the unsorted portion of the array. An additional optimization was made by avoiding unnecessary swaps when the minimum element was already in its correct position. The corrected algorithm now produces accurate sorting results, performs fewer comparisons and swaps, maintains **$O(n^2)$** time complexity with **$O(1)$** extra space, and is more efficient than the original implementation.